Reviewer Pack — Minimal Rank of Brauer Groups and Communication Complexity R…

Entry b1f2334d5a4a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 20:10:53 UTC

Minimal Rank of Brauer Groups and Communication Complexity Rank Inequality

Entry ID: `b1f2334d5a4a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 20:10:53 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Brauer Group Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For all Boolean functions f, the rank of its associated representation in the Brauer group B_f is linearly correlated with its communication complexity rank r_f, such that B_f = Θ(r_f).

Rationale (proposer's reasoning):

> The Brauer group captures the algebraic structure of vector bundles over finite fields, which has been used to study entanglement in quantum information theory. If this structure can be related to communication complexity, it may provide new insights into the hardness of distributed computation.

Taxonomy category: `BrauerGroupToCommunicationComplexityRank` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9c37510c887c2981`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between the ranks of Brauer groups and communication complexity for all tested Boolean functions is greater than or equal to 0.7, with p-value ≤ 0.05, across at least 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Brauer group theory" AND "communication complexity rank" - "minimal rank of Brauer groups" AND matrix rank" - "Boolean functions" AND ("Brauer group" OR B_f) AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10  # Start with a small size and increase if needed
    instances_tested = 0
    total_brauer_rank = 0
    total_comm_complexity_rank = 0

    while instances_tested < 30:
        f = [random.choice([0, 1]) for _ in range(2**n)]

        # Compute Brauer group rank (simplified example)
        brauer_rank = sum(f) % n

        # Compute communication complexity rank (simplified example)
        comm_complexity_rank = max(f.count(0), f.count(1))

        total_brauer_rank += brauer_rank
        total_comm_complexity_rank += comm_complexity_rank
        instances_tested += 1

    mean_brauer_rank = total_brauer_rank / instances_tested
    mean_comm_complexity_rank = total_comm_complexity_rank / instances_tested
    correlation_coefficient = (instances_tested * sum(brauer_rank * comm_complexity_rank for braue
                               mean_brauer_rank * instances_tested * mean_comm_complexity_rank) /
        math.sqrt(instances_tested * sum((brauer_rank - mean_brauer_rank)**2 for brauer_rank in ra
                  instances_tested * sum((comm_complexity_rank - mean_comm_complexity_rank)**2 for

    return {
        "metric_name": "Pearson correlation coefficient",
```

```
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n,
        "conjecture_holds": correlation_coefficient >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = f"Brauer rank {result['metric_value']}, Comm complexity rank {res
                print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={
                break
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
 30, 'n_max': 10, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.15340773408139785, 'ins
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.14859473930208095, 'ins
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.1139067072872042, 'inst
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.15159080890227208, 'ins
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.15413583372262868, 'ins
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.14464271538348997, 'ins
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.10093465148718993, 'ins
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.14352385755966385, 'ins
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.1435754924687911, 'inst
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': -0.13890096923122736, 'ins
RESULT: FALSIFIED counterexample="Brauer rank -0.1301963498018696, Comm complexity rank 30" first_fai
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test uses a simplified model for computing the Brauer group rank and communication complexity rank, which may not accurately represent the actual mathematical concepts.

The Brauer group is a complex algebraic structure that cannot be represented by a simple sum modulo n, and communication complexity is a measure of the minimum number of bits required to communicate information between parties, which also cannot be captured by counting zeros or ones in a list.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The Pearson correlation coefficient between the ranks of Brauer groups and communication complexity for any seed was less than 0.5, which is below the | next: Investigate alternative methods to compute the Brauer group rank that may provide a more accurate representation of the actual mathematical concepts.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12286
2	propose	ollama_remote	glm4:latest	0	0	12797
3	preregistration	ollama_remote	glm4:latest	0	0	12158
4	novelty	ollama_remote	glm4:latest	0	0	8480
5	novelty	ollama_remote	glm4:latest	0	0	14046
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25543
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11677
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12603
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9456
10	critic	ollama_remote	glm4:latest	0	0	52538
11	judge	ollama_remote	glm4:latest	0	0	9410

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 180996 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/b1f2334d5a4a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b1f2334d5a4a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b1f2334d5a4a.tar.gz` (if generated)